

WesCodies — The Round Table

An Autonomous Options-Trading Agent for the Alpaca AI Trading Agents Hackathon

Project Scope, Architecture & 7-Day Build Plan

Team size	2 builders
Timeline	Aug 28 – Sep 4 2026 (7 days)
Foundation	Existing SMC/ICT signal-detection engine (forex bot), generalised to equities & options
Core mechanism	Every proposed trade is cross-examined by an independent agent before execution — “The Round Table”
Autonomy	Fully autonomous — no human approval step, per hackathon core requirements
Hosting	Vercel (frontend) · Railway (backend, worker, MCP sidecar) · Railway/Neon Postgres (database)

The signal-detection engine already exists and already works. What this build adds is a deliberation layer — The Round Table — that gives every proposed trade an independent second opinion before capital moves, and a home for the whole pipeline to run autonomously against Alpaca's paper environment.

1. Project Overview

WesCodies — The Round Table is an autonomous trading agent built on top of an existing Smart Money Concepts (SMC/ICT) signal-detection engine. The engine already detects Order Blocks, Fair Value Gaps, liquidity grabs and CHoCH/BOS structure from raw price action — that part is proven and does not need to be rebuilt. What's new for this hackathon is the deliberation and execution layer around it: every signal the detector proposes is independently cross-checked before an order is ever placed, and the whole loop runs unattended against Alpaca's paper-trading environment with options as the execution vehicle.

Why a Round Table, not just a signal bot

A rules-based detector is consistent but not self-critical — it applies the same logic whether the setup is clean or marginal, and it never asks whether the picture it built is missing something (an earnings print, an options skew signal, a contradicting higher-timeframe read). The Round Table exists to supply that missing second opinion mechanically, on every single trade, with no human in the loop. It doesn't make the system "intelligent" in any deeper sense — it is still a functional agent, not a conscious one — it simply forces a structured, independent challenge before capital is risked.

Hackathon fit

Requirement	How this project satisfies it
Autonomous agent, Alpaca Trading API	Signal → Round Table → Execution runs end-to-end with no manual approval step
MCP server or CLI	Order execution is routed through Alpaca's MCP server (see Section 4), not a raw S
Options trading	The Execution Agent selects and places the options structure (long call/put or defin
Fresh \$100k paper account	A new dedicated Alpaca paper account is created on Day 1, used only for this even

2. Pipeline Architecture

Four stages, each a separate function/process boundary so isolation is enforced in code, not just in the prompt. No stage waits on a human.

Stage	Component	Role
1	Signal Agent	The existing SMC detector (<code>smc_detector.py</code>), generalised via an Instrument abstraction to work on equities/ETFs instead of only forex. Scans the watchlist and proposes a structured trade: direction, entry, stop, target, confluence factors, reasoning.
2	Independent Market Agent (inside the Round Table)	A separate function call that receives only the symbol — never the Signal Agent's proposal or reasoning. Pulls its own read: trend/structure context, options chain IV/skew, any earnings or news catalyst. Produces an independent directional view.
3	Risk-Gate Agent (inside the Round Table)	Compares the Signal Agent's proposal against the Independent Market Agent's read. Flags contradiction, cherry-picked evidence, or overconfidence. Returns a verdict — approve / downsize / reject — plus the options structure that fits the conviction level.
4	Execution Agent	Consumes the verdict programmatically. Approve/downsize selects the contract (expiry, strike) and places the order through Alpaca's MCP server. Reject logs the skip with reason — nothing is sent to Alpaca.

Closing the loop

- Every closed trade (outcome + P&L) is written back to Postgres along with the Round Table's original verdict and bias flags — not just the price-action outcome.
- The adaptive learner mines this history for two kinds of pattern: the original price-action patterns (confluence score, session, setup type) plus a new category — *does a given bias flag actually predict a loss?* That second category is what lets the system tune the Round Table itself, not just the detector's parameters.
- This is the "drastic improvement" story for the pitch: the gate doesn't just block bad trades once, it gets measurably better at knowing which of its own flags are worth trusting.

3. Full Tech Stack

Layer	Choice	Notes
Signal detection	Existing smc_detector.py + Instrumentation	Reusable as-is; pip/lot-specific math replaced with tick-size
Backend API	Python + FastAPI	Serves the dashboard, exposes trade/verdict history, health check
Agent orchestration	LangGraph	3-node graph for the Round Table (Independent Market Agent →
LLM reasoning	Groq (fast calls) + Gemini 2.5 Flash (reasoning) + Risk Gate (app)	Free tier, integrated through Gate (app) AI-compatible client with a pro
Broker / execution	Alpaca Trading API via Alpaca Model	Order placement, options chain lookup, and account state all rout
Database	PostgreSQL	Trades, Round Table verdicts + bias flags, outcomes, adaptive-le
Frontend	Next.js + TypeScript	Live trade feed, Round Table deliberation view (the demo's best v
Background worker	Long-running Python process (Docker)	Polls the watchlist on a schedule (5–15 min), runs the full pipeline
CI/CD	GitHub → auto-deploy on push	Vercel builds the frontend; Railway rebuilds backend/worker conta

4. Hosting & Deployment

The one component that shapes the hosting choice is the always-on background worker — the trading loop has to keep running unattended between judging checks, which rules out pure serverless functions for that piece. The rest of the stack is split across three platforms, each doing the thing it's actually built for.

Component	Host	Why
Frontend (Next.js dashboard)	Vercel	Purpose-built for Next.js — zero-config GitHub deploys, free tier
Backend API (FastAPI)	Railway	Docker-native, persistent web service (not cold-start serverless), simple
Trading-loop worker	Railway (separate background worker service in the same project)	Needs to run continuously, independent of API request traffic. Railway
Alpaca MCP server	Railway (sidecar container, same private network as backend)	Keep private MCP work close to backend latency; the Execution Agent calls
Database	Railway Postgres (managed add-on)	Co-located with the backend/worker on the same private network & shared

Data flow at a glance

Worker (Railway) polls market data → Signal Agent proposes a trade → Round Table deliberates (LLM calls to Groq/Gemini) → Execution Agent calls the Alpaca MCP server (Railway, private network) → order lands in the Alpaca paper account → outcome + verdict written to Postgres (Railway) → FastAPI (Railway) serves that history → Next.js dashboard (Vercel) renders it live for the demo.

Secrets & environment

- Alpaca API keys, LLM provider keys, and the Postgres connection string live only in Railway's environment-variable store — never committed to the repo, never in the Vercel frontend build (the frontend only talks to the FastAPI backend, never to Alpaca or the LLM providers directly).
- A single docker-compose.yml mirrors the Railway service layout (api, worker, mcp, postgres) for local development, so the team can test the full loop before every push.
- The dedicated hackathon paper account's ID is captured in the write-up per the submission requirements, once created on Day 1.

5. Day-by-Day Milestones

Two vertical-slice tracks: **Person A** owns the Signal/Round Table/Execution pipeline, **Person B** owns hosting, database, and the frontend dashboard. Both tracks touch the same repo from Day 1 so deploys are never a Day-7 surprise.

Day 1 (Fri 28 Aug) — Accounts, hosting skeleton, watchlist

Owner: Both

- Register on lablab.ai + Discord. Create the fresh, dedicated \$100k Alpaca paper account.
- Stand up empty services: Vercel project (frontend), Railway project with api/worker/postgres services wired together.
- Get one manual order round-tripping through the Alpaca MCP server end-to-end.
- Pick the equity/ETF watchlist (5–10 liquid, optionable tickers).

Day 2 (Sat) — Instrument abstraction + Signal Agent on equities

Owner: Person A: detector generalisation · Person B: Postgres schema + Railway service wiring

- Generalise smc_detector.py via the Instrument abstraction (tick size, market hours, dollar-based risk sizing replacing pip/lot math).
- Confirm the detector fires sensible signals on the watchlist via backtest before building anything on top of it.

Day 3 (Sun) — Round Table — Independent Market Agent + Risk-Gate Agent

Owner: Person A: agent logic · Person B: FastAPI endpoints + dashboard skeleton on Vercel

- Build both agents as isolated function calls; wire into LangGraph.
- Verdict schema (approve/downsize/reject + bias flags) lands in Postgres.

Day 4 (Mon) — Execution Agent + options contract selection

Owner: Person A: execution + contract logic · Person B: live trade feed UI

- Contract selection (expiry, strike) mapped to verdict/conviction level; order placement via MCP.
- First fully autonomous cycle: signal → gate → real paper order, no human step.

Day 5 (Tue) — Autonomous loop live + adaptive learner

Owner: Person A: adaptive learner · Person B: performance dashboard + Round Table deliberation view

- Background worker running continuously on Railway, scanning on schedule.
- Extend the adaptive learner with Round-Table-aware insights (bias-flag predictiveness, verdict accuracy).

Day 6 (Wed) — Integration, polish, build-in-public

Owner: Both: integration day

- Full dashboard wired to live data; Round Table view is the demo's best visual moment — invest time here.
- Post build-in-public updates tagging lablab.ai and Alpaca.

Day 7 (Thu, due 5PM SAST) — Write-up, video, submission

Owner: Both: rehearsal + submission

- One-page write-up: AI logic, risk gates, Alpaca infrastructure.
- Record demo video, finalise repo + README, submit with account ID and social links.

A working, fully-autonomous loop that clearly demonstrates the Round Table catching a bad signal beats an ambitious pipeline that only half-runs by submission time. If the schedule slips, cut the adaptive-learner enhancements (Day 5) before cutting the Round Table itself — the gate is the whole point of the project.